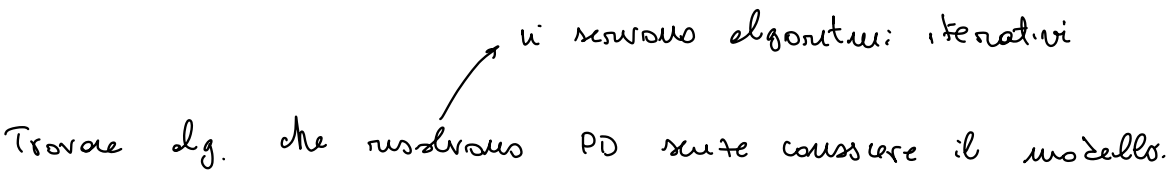
Temporal-difference learning e Q-learning

a.a. 2023/2024

Mario Sassano

Dipartimento di Ingegneria Civile e Ingegneria Informatica
Università di Roma Tor Vergata

Nelle lezioni precedenti...



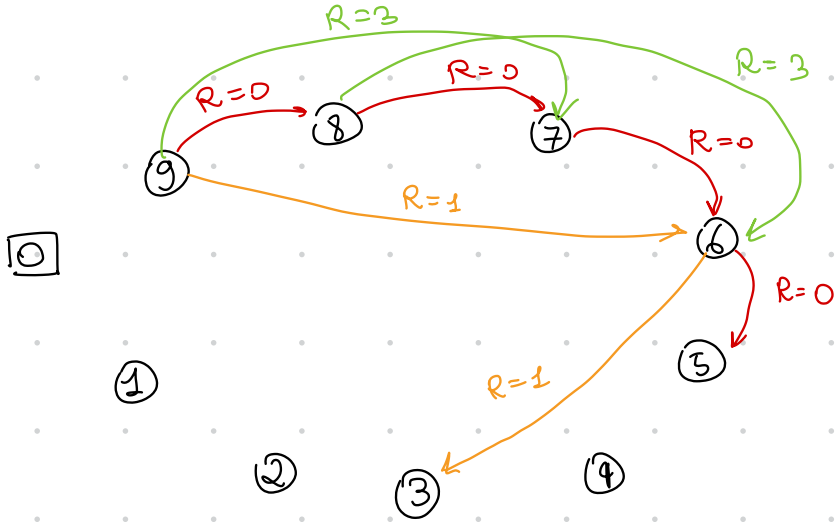
- Abbiamo introdotto la definizione di Processo Decisionale Markoviano stocastico
- Abbiamo studiato le tecniche di *Policy Evaluation* e *Policy Improvement* per valutare e migliorare, rispettivamente, il ricavo ottenuto seguendo una policy
- Abbiamo visto che anche troncando PE ad un numero finito di iterazioni otteniamo convergenza alla soluzione ottima (*Value Iteration*)

Ricostruiamo il knapsack come processo decisionale markoviano e quindi risolverlo con Value Iteration; in questo caso conosco il modello ma SARSA e Q-LEARNING funzionerebbero

Dobbiamo definire: X, U, T, R

$X := \{\text{spazio libero}\} = \{0, 1, 2, \dots, 9\}$, $0 \in X^+$

$R_{k+1} = e(x_k, u_k)$



$U := \{\text{aggiungo vuoto, aggiungo cubo, aggiungo attrezzatura}\}$

	VOLUME	VALORE
spazio vuoto	1	0
cubo	2	2
attrezzatura	3	1

Andiamo la politica $\pi(x) = \text{vuoto} \quad \forall x \in X$

$T_{vuoto} =$

	0	1	2	...	g
0	0	0	0	...	0
1	1	0	0	...	0
2	0	1	0	...	0
⋮			⋱		
g				1	

$R_{vuoto} =$

*
0
0
⋮
0

T_{cibo}

	0	1	2	...	g
0	0	0	0	...	0
1	*	*	*	...	0
2	1	0	...		0
⋮	0	1	...		0
g			⋱		

$R_{cibo} =$

*
*
2
⋮
2

Facciamo value iteration

$$V_{i+1}(x) = \max_u \left\{ r(x,u) + \gamma \underbrace{V_i(x')}_{TV_i(x)} \right\}$$

Temporal-difference learning

$V(x_N)$ è la stima della fine valore

Temporal-difference (TD) learning, idea centrale del Reinforcement Learning:

- può imparare direttamente dall'esperienza senza modello (*model-free*)
- aggiorna i valori correnti anche sulla base di informazioni stimate (*bootstrapping*)

è quello che fa Dijkstra (stime successive)

AGGIORNO STIME CON STIME

Data esperienza (ricompense ricevute) seguendo una policy π, si aggiorna la stima del valore di uno stato intermedio:

- metodi **Monte-Carlo**: si aspetta la fine dell'episodio, si raccoglie il "vero" ricavo ottenuto da x_k, ovvero $G_k = \sum_{i=0}^{N} \gamma^i R_{k+i+1}$, e si usa tale valore come *target* per aggiornare la stima dello stato x_k

somma delle ricompense ottenute

(*)

converge per un # suff. elevato di iterazioni.

$$V(x_k) \leftarrow V(x_k) + \alpha[G_k - V(x_k)], \qquad \alpha \in (0,1)$$

STIMA DELLA F.NE VALORE.

=> TROVO UN VALORE ATTESO

- metodi **Temporal Difference**: si aspetta solo l'istante successivo e si usa la *sua stessa stima attuale* come target[1]

AGGIORNO UNA STIMA CON UNA STIMA

$$V(x_k) \leftarrow V(x_k) + \alpha[R_{k+1} + \gamma V(x_{k+1}) - V(x_k)]$$

[1] via di mezzo tra MC e TD: consideriamo M reward successivi, $R_{k+1} + \ldots + \gamma^M R_{k+N} + \gamma^{N+1} V(x_{k+N}) \rightarrow N-stepTD$, Temporal Difference con ricompense raccolte su M passi

$V(x_n)$ è la mia stima

G_n è il valore reale (simulato)

↓

quanto si discosta $V(x_n)$ da G_n che però una realizzazione del processo.

Devo simulare tutto $\left\{\begin{array}{l} \text{Un solo } G_n \text{ dipende anche dalla lunghezza} \\ \text{dell'episodio} \Rightarrow \text{il calcolo devo farlo} \end{array}\right.$

Come faccio dunque a conoscere G_n senza calcolarlo (tutto) $\Rightarrow$ stimo G_n solo ad un passo.

$$V(x_n) \leftarrow V(x_n) + \alpha \left[\underbrace{R_{n+1}}_{\text{info reale}} + \gamma V(x_{n+1}) - V(x_n) \right]$$

} definizione di Bootstrapping

$\gamma V(x_{n+1})$ ↓ è una stima del valore che ho in $V(x)$.

(*) Nel M.C. se non ho un ist. terminale devo troncare il calcolo della ricompensa

Monte-Carlo vs TD

Problema: tornare a casa dall'università

⇒ le ricompense corrispondono al tempo trascorso (con il segno cambiato...)

⇒ stati: lasciare l'università (martedì alle 16), arrivare alla macchina (piove), lasciare il GRA (nessun incidente), strada interna (dietro al camion), strada di casa, arrivare a casa

Non ho stati da predire, osservo il processo e stimo la fine reale.

stato	tempo trascorso	stima tempo rimasto	stima tempo totale
1	0	30	30
2	5	35	40
3	20	15	35
4	30	10	40
5	40	3	43
6	43	0	43

⇒ ARBITRARIO BASATO PER ES. SU SERIE STORICHE

REALE

ho info. sulle realizzazione in termini di Reward (st. (tempo trascorso)

Vediamo come Monte-Carlo e TD avrebbero aggiornato le stime...

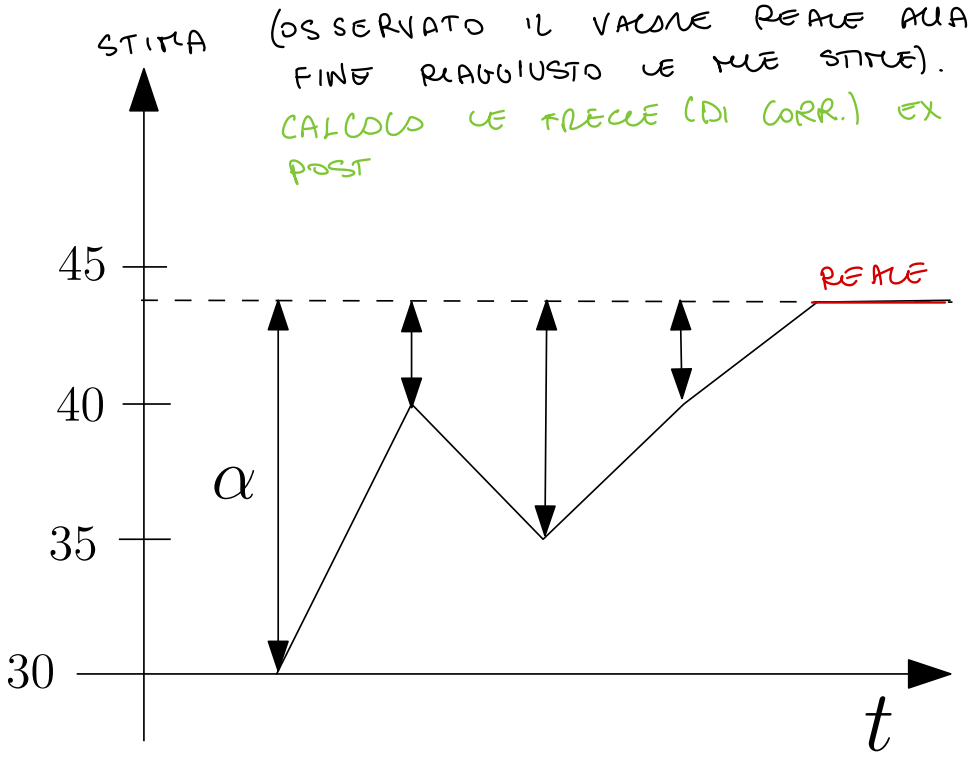
Passo da uno stato al succ. e voglio stimare il tempo rimasto per tornare a casa.

In 1 avevo una stima di 30 min
In 2 ho un tempo trascorso dall'ist. iniziale $\geq$ 5 min (è un tempo aggiuntivo) a cui sommo il tempo stimato per tornare a casa (35 min).

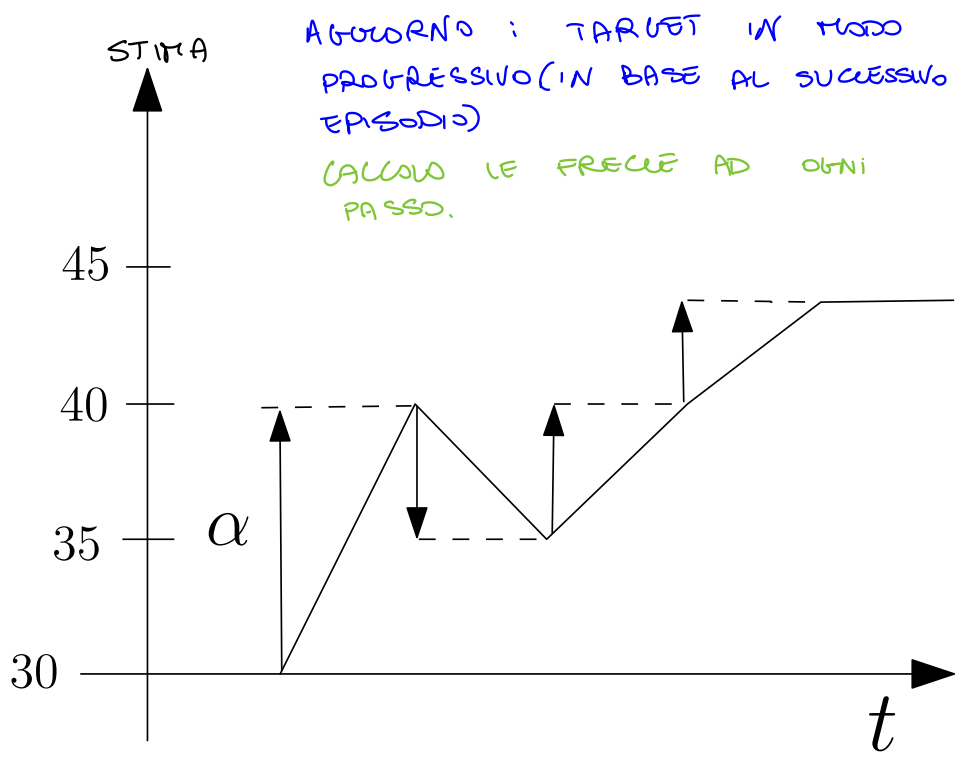
Problema: tornare a casa dall'università

⇒ le ricompense corrispondono al tempo trascorso (con il segno cambiato...)

⇒ stati: lasciare l'università (martedì alle 16), arrivare alla macchina (piove), lasciare il GRA (nessun incidente), strada interna (dietro al camion), strada di casa, arrivare a casa



Aggiornamento Monte-Carlo



Aggiornamento TD

Commenti su TD learning

$$V(x_k) \leftarrow V(x_k) + \alpha_k [R_{k+1} + \gamma V(x_{k+1}) - V(x_k)]$$

- si può dimostrare che la stima converge al valore vero $\sqrt{}(x)$, a patto che il passo di aggiornamento (tempo-variante) α_k soddisfi

$$\left\{ \sum_{k=1}^{\infty} \alpha_k = \infty, \quad \sum_{k=1}^{\infty} \alpha_k^2 < \infty \right\}$$

FARE LA DIMOSTRAZIONE SUL LIBRO DI SILVER

$\Rightarrow$ ad esempio, $\alpha_k \propto \frac{1}{k}$

- Cerchiamo di usare TD per il **controllo**

  $\Rightarrow$ possiamo usare lo schema di PI con la fase di *evaluation* eseguita con TD...

  $\Rightarrow$ ...oppure possiamo stimare direttamente la funzione $q_\star$

come calcolo l'arg-max se non conosco il modello.

posso presumere dal modello, continuamente stimando e scegliendo l'azione.

Punti aperti:

- exploration vs exploitation
- on-policy vs off-policy

On-policy TD: SARSA

Con TD stimiamo direttamente $q_\pi(x, u)$ invece di $v_\pi(x)$

Devo visitare x_k, u_k tante volte per convergere a $q_*(x_k, u_k)$

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha \left[R_{k+1} + \gamma Q(x_{k+1}, u_{k+1}) - Q(x_k, u_k) \right]$$

La legge di aggiornamento utilizza tutti gli elementi della quintupla:

$$(x_k, u_k, R_{k+1}, x_{k+1}, u_{k+1})$$

Stato - Azione - Ricompensa - Stato - Azione

STARTING POINT — SCELGO — OSSERVO — OSSERVO — SCELGO

Algoritmo: come faccio bootstrapping

- ci troviamo in uno stato x_k
- scegliamo un'azione u_k in base a Q $\Rightarrow u_k \in \arg\max_u Q(x_k, u)$ (la scelta migliore è $\arg\max q(x_k, u)$ ma qui ho solo una stima)
- riceviamo una ricompensa R_{k+1} e andiamo a finire in x_{k+1}
- scegliamo un'azione u_{k+1} in base a Q $\Rightarrow u_{k+1} \in \arg\max_u Q(x_{k+1}, u)$
- aggiorniamo la stima di Q con TD

$\Rightarrow$ se x_{k+1} è uno stato terminale poniamo $Q(x_{k+1}, u_{k+1}) = 0$ (stima e valore vero coincidono. $= q(x_{k+1}, u_{k+1})$.)

$\Rightarrow$ se scegliamo sempre in base a Q non rischiamo di fare sempre le stesse azioni? in base ad una stima

Exploration vs Exploitation

Due modi per garantire di non fare sempre le stesse azioni:

- **exploring start**: si deve garantire che ciascuna coppia di stato e azione sia selezionata come la prima (start) di un episodio un numero infinito di volte

- ε-**greedy policy**: senza *exploring start* una policy deterministica (greedy rispetto alla stima di Q) ci porterebbe sempre alle stesse azioni (ritenute più remunerative in quel momento..)

  $\Rightarrow$ policy non-deterministica $(0 < \varepsilon << 1)$:

  - con probabilità $1 - \varepsilon$ scegliamo u_k in maniera greedy rispetto a Q
  - con probabilità ε scegliamo u_k casualmente

$\Rightarrow$ Suggerimento: ε potrebbe variare nel tempo con $\lim_{k\to\infty} \varepsilon_k = 0$... in modo da convergere rispetto alle scelte greedy

Se $|X| = 10^6$ $\wedge$ $|U| = 100$ dovrei visitare ∞ volte ogni el $\Rightarrow$ sono 10^8 el. aggiornati ∞ volte.

LEGGE TABELLARE

Meno quantità (misura $|X|$) più l'algoritmo è leggero $\Rightarrow$ vedi es. dei telefoni con livello "basso", "medio", "alto", "vuoto".

↳ ∃ un trade-off tra dim. matrice e descrizione del sistema.

Off-policy TD: Q-learning

Se conosciamo per ciascuno stato il valore di tutte le possibili azioni (e non solo quelle che siamo esplorando)...

tra tutte le scelte aleatorie possibili prendo quella che porterebbe al massimo (*)

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha \left[R_{k+1} + \gamma \max_u Q(x_{k+1}, u) - Q(x_k, u_k) \right]$$

$\Rightarrow$ Q-learning, uno dei metodi più diffusi di RL

$\Rightarrow$ implementazione *off-line* anche solo su dati precedentemente raccolti (ad esempio migliaia di partite di scacchi, GO...)

(*) Il ragionamento funziona solo se ho un quantitativo di dati in modo da stimare il valore del massimo.

RL con approssimatori funzionali: Deep Reinforcement Learning

Fino ad ora abbiamo visto metodi con *tabelle*: → andiamo oltre SARSA e Q-LEARNING eliminando le tabelle.

- un valore per ciascuno stato e/o azione
- aggiornamenti continui e individuali

⇒ impossibile se ho un numero enorme di stati (ad esempio nel GO..)!

Uso di Approssimatore funzionale (di una funzione non nota f) o troppo complicata

data una collezione di coppie $x_i, y_i = f(x_i),\ i = 1, ..., N$

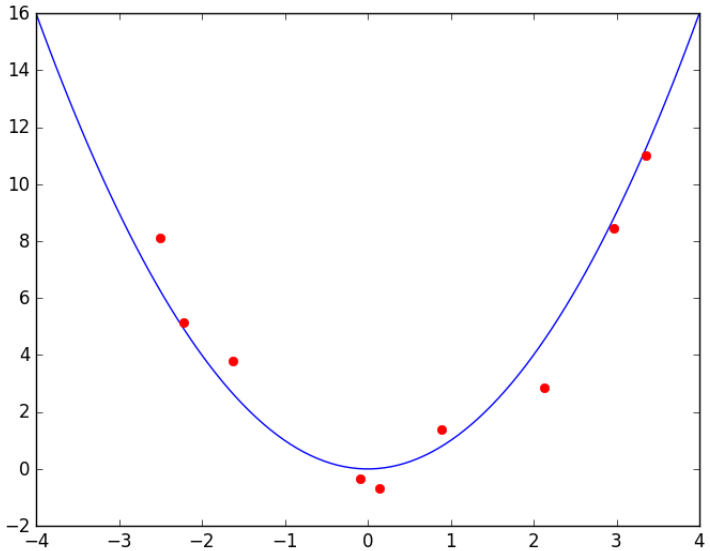
consideriamo una funzione $s(x, w)$ dipendente da un numero ristretto di *coefficienti* w

→ esempio: $s(x, w) = w_0 + w_1 x + w_2 x^2$

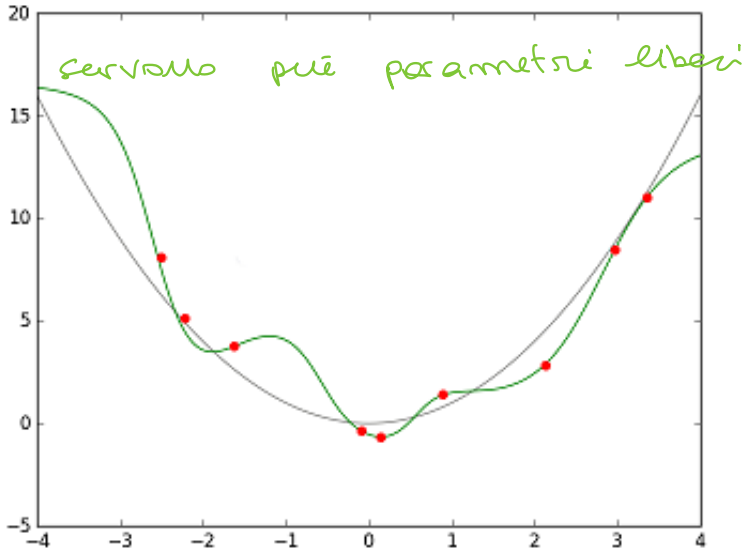
⇒ determinare il valore di w tale che $s(x_i, \bar{w})$ *approssimi* o *interpoli* y_i per ogni i

⇓

Rete Neurale!



approssimazione



interpolazione

Uso di Approssimatore funzionale (di una funzione non nota f)

data una collezione di coppie $x_i, y_i = f(x_i),\ i = 1, ..., N$

consideriamo una funzione $s(x, w)$ dipendente da un numero ristretto di *coefficienti* w

$\rightarrow$ esempio: $s(x, w) = w_0 + w_1 x + w_2 x^2$

$\Rightarrow$ determinare il valore di w tale che $s(x_i, \bar{w})$ *approssimi* o *interpoli* y_i per ogni i

Policy Evaluation con approssimatori funzionali

La funzione valore non è più rappresentata da una "tabella" ma da una funzione parametrizzata[2] *da un vettore di coefficienti* $w \in \mathbb{R}^n$

$$\hat{v}(x, w) \approx v_\pi(x)$$

Aggiornamento *tabellare* del solo stato x $\Rightarrow$ aggiornamento dei pesi w

combinazione delle fm: note

- Supponiamo di avere una stima V_k del valore dello stato corrente
- Aggiorniamo i pesi per minimizzare l'errore quadratico

$$w_{k+1} = w_k - \frac{\alpha}{2} \nabla_w \left[V_k - \hat{v}(x_k, w_k) \right]^2$$

$$= w_k - \alpha \left[V_k - \hat{v}(x_k, w_k) \right] \nabla_w \hat{v}(x_k, w_k)$$

- Ad esempio, approssimazione *bootstrap* $V_k = R_{k+1} + \gamma \hat{v}(x_{k+1}, w_k)$

definisce la propagazione

[2] Ad esempio, radial basis functions $\hat{v}(x, w) = \sum_{i=1}^{N} w_i e^{-\alpha_i \|x - x_i\|^2}$

Da x_k percepisco V_k e lo uso per aggiornare tutti i pesi della f.ne => propago l'aggiornamento tramite l'approssimatore lineare

Deep RL => non faccio aggiornamenti tabellari ma pesi di un approssimatore lineare che propaga l'aggiornamento ai vicini (uso f.ni che si muovono su max e minimi e riducono l'effetto della propagazione).